



École Nationale Supérieure
d'Informatique et d'Analyse des Systèmes
- Rabat

Semestre 4 : MLops

Industrialisation d'un Système de Détection d'Anomalies Dentaires : une Approche MLOps de bout en bout

Élèves :
Jad Falaq

Enseignant :
Mr Mohamed Lazzar

Jury :
Pr. Abdellatif el afia
Pr. Raddouane CHIHEB

Année Universitaire 2025-2026

Table des matières

Liste des Abréviations **3**

Résumé **5**

Abstract **6**

1 Introduction Générale **7**
1.1 Contexte 7
1.2 Ce que nous avons construit 7
1.3 Pourquoi ce projet est un projet MLOps 8
1.4 Organisation du Rapport 8

2 Contexte Théorique et Technologies Utilisées **9**
2.1 Le MLOps : pourquoi ce concept existe 9
2.2 Les quatre piliers du MLOps 10
2.3 La détection d'objets appliquée à l'imagerie dentaire 11
2.4 Stack technologique 11
2.5 Architecture globale du système 12

3 Données et Préparation **14**
3.1 Le corpus de radiographies panoramiques 14
3.2 Audit du dataset : ce que nous avons trouvé 14
3.3 La taxonomie initiale et ses limites 15
3.4 Restructuration vers 7 classes cliniques 16
3.5 Le pipeline DVC 18
3.6 Résultat : un dataset propre et documenté 18

4 Modélisation et Expérimentation **20**
4.1 Approche expérimentale 20
4.2 Configuration de l'entraînement 20
4.3 Le suivi des expériences avec MLflow 21
4.4 Résultats et comparaison des variantes 23
4.5 Sélection du modèle Champion 24

5 Déploiement et Industrialisation **26**
5.1 De l'expérience au service 26
5.2 L'API REST 26
5.3 L'interface utilisateur 27
5.4 La conteneurisation avec Docker 28

5.5 Les pipelines CI/CD 29
5.6 L'infrastructure cloud 30
Liens du projet 32

6 Conclusion et Perspectives **33**
6.1 Ce que ce projet a réellement accompli 33
6.2 Ce qui a bien fonctionné 34
6.3 Les limites du système 34
6.4 Perspectives d'évolution 34
6.5 Réflexion finale 35

Liste des Abréviations

Abréviation	Signification
API	*Application Programming Interface* — Interface de programmation applicative permettant la communication entre composants logiciels
bbox	*Bounding Box* — Boîte englobante délimitant un objet détecté dans une image
CD	*Continuous Deployment* — Déploiement continu automatisé
CI	*Continuous Integration* — Intégration continue du code avec vérification automatisée
CLI	*Command Line Interface* — Interface en ligne de commande
CPU	*Central Processing Unit* — Unité centrale de traitement
DVC	*Data Version Control* — Outil de versionnage des données et des pipelines ML
EDA	*Exploratory Data Analysis* — Analyse exploratoire des données
ENSIAS	École Nationale Supérieure d'Informatique et d'Analyse des Systèmes
GPU	*Graphics Processing Unit* — Unité de traitement graphique utilisée pour l'accélération du calcul ML
HTTP	*HyperText Transfer Protocol* — Protocole de communication du web
IoU	*Intersection over Union* — Mesure de chevauchement entre deux boîtes englobantes, utilisée pour évaluer la qualité des détections
JSON	*JavaScript Object Notation* — Format léger d'échange de données
mAP	*mean Average Precision* — Précision moyenne sur l'ensemble des classes, métrique principale en détection d'objets
ML	*Machine Learning* — Apprentissage automatique
MLOps	*Machine Learning Operations* — Discipline combinant ML, génie logiciel et DevOps pour industrialiser les systèmes ML

Abréviation	Signification
NMS	*Non-Maximum Suppression* — Algorithme de post-traitement éliminant les détections redondantes
OPG	*Orthopantomogramme* — Radiographie panoramique dentaire
PFA	Projet de Fin d'Années
REST	*Representational State Transfer* — Style d'architecture pour les services web
URL	*Uniform Resource Locator* — Adresse web identifiant une ressource sur internet
YAML	*YAML Ain't Markup Language* — Format de sérialisation de données lisible par l'humain, utilisé pour les fichiers de configuration
YOLO	*You Only Look Once* — Famille d'architectures de détection d'objets en temps réel
YOLOv8	Huitième génération de l'architecture YOLO, développée par Ultralytics

Résumé

La santé bucco-dentaire représente un enjeu de santé publique majeur à l'échelle mondiale. Selon l'Organisation Mondiale de la Santé, les maladies bucco-dentaires touchent près de 3,5 milliards de personnes, dont la majorité dans des pays à ressources limitées où l'accès aux soins dentaires spécialisés reste insuffisant. Dans ce contexte, l'imagerie dentaire panoramique constitue l'un des outils diagnostiques les plus utilisés en pratique clinique : elle offre en une seule acquisition une vue complète de la dentition, des structures osseuses et des pathologies associées. Cependant, l'interprétation de ces radiographies demeure une tâche longue, exigeante et soumise à la variabilité inter-praticien, ce qui peut engendrer des erreurs de diagnostic ou des retards de prise en charge.

Ce projet de fin d'année s'inscrit dans cette réalité clinique avec une ambition claire : concevoir et déployer un système intelligent d'aide à la détection automatique d'anomalies dentaires, capable de fonctionner à grande échelle et d'être intégré dans un flux de travail clinique réel. Le système développé est entraîné à reconnaître sept catégories d'anomalies et de structures dentaires parmi les plus fréquentes en pratique : les lésions carieuses, les pathologies périapicales, les pertes osseuses parodontales, les dents incluses, les pathologies radiculaires, les dents traitées et les dispositifs implantaires. Ces catégories ont été définies en collaboration avec la taxonomie clinique existante, en regroupant des classes initialement trop fragmentées pour être cliniquement exploitables.

Au-delà de la performance du modèle, l'enjeu central de ce travail est d'ordre industriel : comment garantir qu'un système d'intelligence artificielle développé en contexte académique puisse être rendu fiable, reproductible, maintenable et accessible dans un environnement opérationnel ? C'est la question que répond ce projet en adoptant les pratiques du **MLOps**, une discipline qui réconcilie la rigueur de l'ingénierie logicielle avec les exigences spécifiques des systèmes d'apprentissage automatique.

Le résultat de ce travail est un système complet et déployé publiquement, accessible depuis n'importe quel navigateur web, capable d'analyser une radiographie panoramique en quelques secondes et de restituer les zones d'anomalies détectées avec leur localisation précise et leur niveau de confiance. Ce système n'est pas un prototype de laboratoire : il tourne en production sur une infrastructure cloud, avec des pipelines de vérification automatisés garantissant sa qualité à chaque évolution du code.

Mots-clés : Détection d'anomalies dentaires, radiographie panoramique, intelligence artificielle médicale, MLOps, déploiement cloud, aide au diagnostic.

Abstract

Oral health is a major global public health concern. According to the World Health Organization, dental and oral diseases affect approximately 3.5 billion people worldwide, with a disproportionate burden in low- and middle-income countries where access to specialized dental care remains limited. In clinical practice, panoramic dental radiography is one of the most widely used diagnostic imaging tools : it provides a single-acquisition full view of the dentition, bone structures, and associated pathologies. However, the interpretation of these radiographs remains a time-consuming and cognitively demanding task, subject to inter-practitioner variability that can lead to diagnostic errors or delayed treatment.

This final year engineering project addresses this clinical reality with a concrete objective : designing and deploying an intelligent system for the automatic detection of dental anomalies, built to operate at scale and to be integrated into real clinical workflows. The system is trained to identify seven clinically meaningful categories of dental anomalies and structures : carious lesions, periapical pathologies, periodontal bone loss, impacted teeth, root pathologies, treated teeth, and implant devices. These categories were defined by consolidating an initial taxonomy of fourteen annotation classes into a clinically actionable and statistically robust scheme.

Beyond model performance, the central challenge of this work is an engineering one : how can an artificial intelligence system developed in an academic setting be made reliable, reproducible, maintainable and accessible in an operational environment ? This is the question this project answers by adopting **MLOps** practices — a discipline that bridges the rigor of software engineering with the specific demands of machine learning systems in production.

The outcome of this work is a fully deployed system, publicly accessible from any web browser, capable of analyzing a panoramic radiograph in seconds and returning detected anomaly zones with precise localizations and confidence scores. This is not a laboratory prototype : the system runs in production on a cloud infrastructure, with automated verification pipelines that continuously ensure its quality with every code change. It demonstrates that artificial intelligence can be brought from experiment to clinical demonstration in a structured, transparent and reproducible manner.

Keywords : Dental anomaly detection, panoramic radiograph, medical artificial intelligence, MLOps, cloud deployment, diagnostic assistance.

Chapitre 1

Introduction Générale

1.1 Contexte

Lors d'une consultation dentaire, le praticien dispose de quelques minutes pour lire une radiographie panoramique qui concentre pourtant une quantité d'information considérable : caries, pertes osseuses, dents incluses, traitements antérieurs, dispositifs implantaires — tout cela sur un même cliché. Cette lecture est exigeante, et le risque d'oublier un détail existe, surtout en fin de journée ou en cabinet chargé.

C'est à partir de cette réalité concrète que ce projet a été construit. L'idée n'est pas de remplacer le dentiste, mais de lui fournir un second regard automatique — un système capable d'analyser la radiographie en quelques secondes et de signaler les zones qui méritent attention. Cette idée n'est pas nouvelle dans le monde médical, mais sa mise en œuvre reste complexe dès qu'on sort du cadre expérimental.

Car là réside le vrai problème : la majorité des projets d'intelligence artificielle médicale fonctionnent bien dans un notebook, mais ne passent jamais en production. Ils restent des démonstrations techniques sans impact réel, faute d'avoir pensé au déploiement, à la maintenance, à la traçabilité des expériences, à la reproductibilité des résultats. Ce projet a été conçu pour éviter précisément ce destin.

Il s'inscrit dans le cadre du module MLOps de l'ENSIAS, dont l'objectif est d'enseigner comment on construit non pas seulement un modèle, mais un *système* — quelque chose qui tourne, qui se maintient, qui se met à jour et qui reste accessible.

1.2 Ce que nous avons construit

Le résultat de ce travail est un service en ligne, accessible depuis un navigateur, qui prend en entrée une radiographie panoramique dentaire et retourne les anomalies détectées avec leur localisation et leur niveau de confiance. Ce service tourne aujourd'hui en production sur une infrastructure cloud et peut être testé à l'adresse suivante :

`https://detection-dentaire-mlops.vercel.app`

Derrière cette interface se cache une chaîne complète : un modèle YOLOv8 entraîné sur un corpus de 1 808 radiographies annotées, une API REST construite avec FastAPI, une image Docker embarquant le modèle, et des pipelines de tests automatisés qui vérifient le bon fonctionnement du système à chaque modification du code.

Le modèle est capable de détecter sept catégories d'anomalies et de structures dentaires, définies à partir d'une taxonomie clinique existante que nous avons rationalisée pour la rendre exploitable par l'apprentissage automatique.

1.3 Pourquoi ce projet est un projet MLOps

Un projet MLOps se distingue d'un projet de Machine Learning classique par ce qu'il fait *après* l'entraînement du modèle. N'importe qui peut entraîner un modèle YOLOv8 en quelques lignes de code aujourd'hui. Ce qui est difficile, c'est de répondre aux questions suivantes :

— Comment s'assurer que les données utilisées sont propres, cohérentes et traçables ?
— Comment comparer objectivement plusieurs variantes de modèles et justifier le choix de l'une d'elles ?
— Comment rendre le service accessible sans dépendre d'un environnement local ?
— Comment garantir qu'une modification du code ne casse pas silencieusement le service en production ?

Ce sont ces questions qui structurent ce projet. Les réponses apportées s'appuient sur des outils standards de l'industrie : DVC pour le versionnement des données, MLflow pour le suivi des expériences, Docker pour la portabilité du service, GitHub Actions pour l'automatisation des vérifications, Railway et Vercel pour le déploiement cloud.

1.4 Organisation du Rapport

Ce rapport suit la progression naturelle du projet, du traitement des données jusqu'au service déployé.

Le **Chapitre 2** pose les bases théoriques : qu'est-ce que le MLOps, pourquoi en a-t-on besoin, et quels outils ont été retenus pour ce projet.

Le **Chapitre 3** traite la dimension données : comment le corpus a été audité, nettoyé, restructuré et versionné avant tout entraînement.

Le **Chapitre 4** couvre la modélisation et l'expérimentation : choix du modèle, entraînement, suivi des expériences avec MLflow, comparaison des variantes et sélection du modèle final.

Le **Chapitre 5** présente le déploiement dans son intégralité : l'API, l'interface utilisateur, Docker, les pipelines CI/CD et les URLs de production.

Le **Chapitre 6** dresse un bilan honnête du projet — ce qui fonctionne, ce qui pourrait être amélioré, et les perspectives d'évolution.

Chapitre 2

Contexte Théorique et Technologies Utilisées

2.1 Le MLOps : pourquoi ce concept existe

Pendant longtemps, les équipes de data science et les équipes d'ingénierie logicielle ont travaillé en silos. Les premiers entraînaient des modèles dans des notebooks, obtenaient de bons résultats, puis passaient la main aux seconds en leur demandant de "mettre ça en production". Ce transfert était systématiquement douloureux. Les modèles dépendaient de versions spécifiques de bibliothèques, de chemins de fichiers locaux, de données qui n'avaient pas été versionnées. Reproduire les résultats d'une expérience passée était souvent impossible. Corriger un bug en production signifiait tout recommencer depuis le début.

C'est pour répondre à ce problème structurel que le **MLOps** a émergé, au croisement du *Machine Learning*, du *génie logiciel* et des pratiques DevOps. Le terme est apparu autour de 2015-2016, popularisé notamment par une étude de Google qui analysait les systèmes ML en production [3]. Cette étude a mis le doigt sur quelque chose d'important : dans un système ML industriel, le code qui concerne réellement le modèle ne représente souvent qu'une infime partie de l'ensemble. Le reste — la gestion des données, la configuration, les pipelines, le monitoring, la reproductibilité — constitue l'essentiel du travail, et c'est précisément là que la plupart des projets échouent.

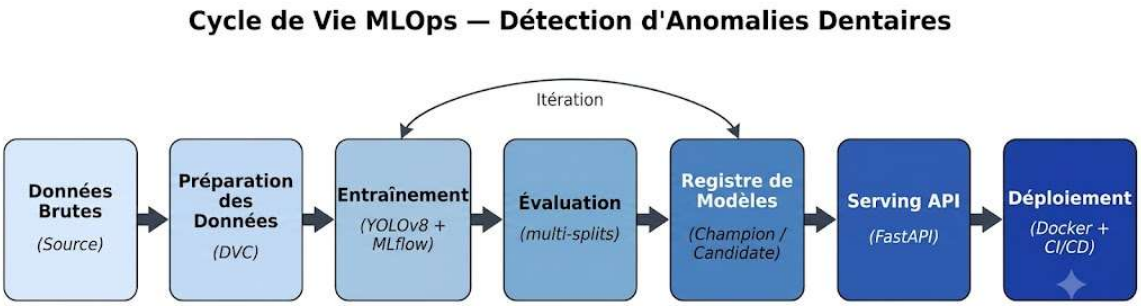


FIGURE 2.1 – Le cycle de vie MLOps : de la donnée brute au service en production

La figure 2.1 illustre ce cycle. On y voit que l'entraînement du modèle n'est qu'une étape parmi d'autres. Ce qui encadre cette étape — la préparation des données en amont, la validation, le déploiement et la vérification en aval — représente en pratique la part la plus importante du travail d'ingénierie.

2.2 Les quatre piliers du MLOps

Quelle que soit l'organisation ou le projet, le MLOps repose sur quatre principes fondamentaux que nous avons appliqués tout au long de ce travail.

La reproductibilité

Un résultat qui ne peut pas être reproduit n'a pas de valeur dans un contexte professionnel. La reproductibilité, en MLOps, signifie que n'importe qui — ou n'importe quelle machine — doit pouvoir obtenir exactement le même résultat en partant du même état initial. Cela implique de versionner non seulement le code, mais aussi les données, les paramètres, et les environnements d'exécution.

Dans ce projet, la reproductibilité repose sur trois mécanismes : DVC pour versionner les données et les pipelines de transformation, un fichier `params.yaml` qui centralise tous les hyperparamètres, et une seed globale fixée à 42 pour toutes les opérations aléatoires.

La traçabilité

Entraîner plusieurs variantes d'un modèle sans système de suivi, c'est naviguer à l'aveugle. Après quelques expériences, on ne sait plus quelle configuration a produit quel résultat, et toute comparaison devient subjective. La traçabilité consiste à enregistrer automatiquement, pour chaque expérience, les paramètres utilisés, les métriques obtenues et les artéfacts produits.

MLflow joue ce rôle dans notre projet. Chaque entraînement ouvre un *run* MLflow qui consigne l'intégralité du contexte de l'expérience. Cette pratique a permis de comparer les variantes de modèles sur des bases objectives et de justifier le choix du modèle final.

La portabilité

Un service qui ne fonctionne que sur la machine de son auteur n'est pas un service. La portabilité signifie que le système doit pouvoir tourner de façon identique sur n'importe quel environnement, sans dépendre de la configuration locale. Docker répond à ce besoin en empaquetant le service et toutes ses dépendances dans une image autonome. Dans notre cas, l'image Docker intègre directement le modèle entraîné, ce qui rend le service entièrement auto-suffisant.

L'automatisation

La vérification manuelle de la qualité d'un système avant chaque déploiement est coûteuse et peu fiable. L'automatisation consiste à définir une fois pour toutes les vérifications à effectuer — tests unitaires, validation des endpoints, contrôle de l'image Docker — et à les exécuter automatiquement à chaque modification du code. GitHub Actions assure cette fonction dans notre pipeline CI/CD.

2.3 La détection d'objets appliquée à l'imagerie dentaire

Pourquoi la détection d'objets et non la classification

Lorsqu'on parle d'intelligence artificielle appliquée à l'analyse d'images médicales, plusieurs niveaux de tâches existent. La **classification** détermine si une image contient ou non une pathologie, mais ne dit pas où elle se trouve. La **segmentation** délimite précisément les pixels appartenant à chaque région d'intérêt, mais requiert des annotations pixel par pixel très coûteuses. Entre les deux, la **détection d'objets** produit des boîtes englobantes qui localisent chaque anomalie dans l'image, avec son étiquette et son score de confiance.

Pour notre cas d'usage, la détection d'objets est le choix naturel. Une radiographie panoramique contient généralement entre dix et seize structures ou anomalies simultanées. Il ne s'agit pas de décider si l'image est pathologique, mais de pointer précisément chaque zone d'intérêt pour que le praticien puisse concentrer son attention.

L'architecture YOLO

La famille YOLO (*You Only Look Once*) est aujourd'hui l'une des architectures de référence pour la détection d'objets en temps réel [2]. Son principe est de traiter l'image en un seul passage du réseau de neurones, en divisant l'image en une grille et en prédisant simultanément les boîtes et les classes pour chaque cellule. Cette approche la rend particulièrement rapide par rapport aux architectures à deux étapes comme Faster R-CNN.

YOLOv8, développé par Ultralytics [1], est la huitième génération de cette famille. Elle introduit une architecture dite *anchor-free* qui prédit directement les coordonnées des centres des boîtes sans utiliser d'ancres prédéfinies, ce qui améliore la précision sur les objets de formes variées et simplifie la configuration. C'est cette version que nous avons retenue, dans sa variante *small* (`yolov8s`), qui offre un bon compromis entre capacité et vitesse d'entraînement.

2.4 Stack technologique

Le tableau 2.1 synthétise l'ensemble des outils utilisés dans ce projet, avec leur rôle précis et la justification du choix.

TABLE 2.1 – Stack technologique du projet

Domaine	**Outil**	**Rôle et justification**
Langage	Python 3.11	Ecosystème ML le plus riche ; requis par toutes les librairies utilisées
Détection	YOLOv8s (Ultralytics)	Architecture moderne, rapide, bien documentée, compatible export Docker
Versionnage données	DVC 3.x	Étend Git aux fichiers binaires ; pipelines reproductibles via `dvc repro`
Tracking	MLflow 2.x	Standard de fait pour le suivi d'expériences ML ; UI de comparaison intégrée
API	FastAPI + Uvicorn	Framework Python le plus rapide pour les APIs REST ; documentation Swagger automatique
Conteneurisation	Docker	Portabilité totale ; image CPU légère pour le déploiement sans GPU
CI/CD	GitHub Actions	Intégré à GitHub ; gratuit pour les projets publics ; déclenchement automatique sur push
Frontend	React 19 + Vite 8	Interface réactive ; preprocessing côté client pour réduire la charge serveur
Déploiement backend	Railway	Déploiement Docker depuis GitHub en un clic ; plan gratuit suffisant pour la démo
Déploiement frontend	Vercel	Optimisé pour les applications Vite/React ; CDN mondial, déploiement automatique
Tests	pytest	Standard Python pour les tests unitaires et d'intégration

2.5 Architecture globale du système

Avant d'entrer dans le détail de chaque composant dans les chapitres suivants, la figure 2.2 donne une vue d'ensemble de la façon dont ces éléments s'articulent.

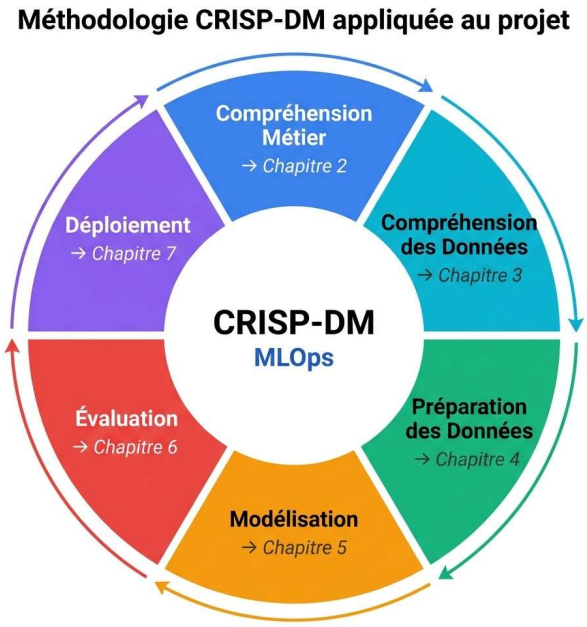


FIGURE 2.2 – Architecture globale du système déployé

On peut distinguer trois couches dans cette architecture. La **couche données** regroupe le corpus brut, le pipeline de transformation DVC et le dataset traité. La **couche modélisation** comprend les expériences MLflow, le modèle Champion sélectionné et le registre de modèles. La **couche service** englobe l'API FastAPI, l'image Docker, le pipeline CI/CD et les applications déployées sur Railway et Vercel. Cette séparation en couches n'est pas qu'une vue de l'esprit : elle correspond à une vraie séparation du code, avec des responsabilités clairement définies entre les modules.

Ce que cette architecture permet, concrètement, c'est qu'un changement dans une couche n'affecte pas les autres. On peut améliorer le modèle sans toucher au service. On peut modifier l'interface utilisateur sans toucher aux données. Et si le service tombe en panne, les pipelines CI/CD le détectent automatiquement et alertent avant qu'un utilisateur ne soit impacté.

Chapitre 3

Données et Préparation

3.1 Le corpus de radiographies panoramiques

Le point de départ de tout projet ML, c'est la donnée. Avant d'écrire une seule ligne de code d'entraînement, nous avons passé un temps considérable à comprendre, auditer et restructurer le dataset. Ce travail est souvent sous-estimé dans les rapports académiques, mais c'est pourtant là que se jouent la plupart des succès ou des échecs.

Le corpus utilisé est un ensemble de radiographies panoramiques dentaires annotées, provenant de cabinets cliniques roumains. Il contient **1 808 images**, réparties en quatre partitions aux rôles distincts :

TABLE 3.1 – Répartition du dataset par partition

Partition	**Images**	**%**	**Rôle**
`train`	1 375	76.0%	Données d'entraînement du modèle
`eval`	153	8.5%	Validation interne pendant l'entraînement
`test`	100	5.5%	Test interne indépendant
`external`	180	10.0%	Test externe, cabinet clinique distinct

La partition `external` mérite une attention particulière. Elle provient d'un cabinet différent de celui qui a fourni les trois premières partitions. Cela signifie que les images ont été acquises avec un équipement différent, par des praticiens différents, sur une patientèle différente. Cette partition est donc un vrai test de généralisation — bien plus exigeant qu'une simple validation croisée interne.

Chaque image est associée à un fichier de labels au format YOLO : une ligne par objet annoté, contenant l'identifiant de classe et les coordonnées normalisées de la boîte englobante. En moyenne, chaque radiographie contient **13.5 objets annotés**, ce qui reflète la densité typique d'une radiographie panoramique en pratique clinique.

3.2 Audit du dataset : ce que nous avons trouvé

Avant tout traitement, nous avons audité systématiquement le dataset avec le script `scripts/inspect_dataset.py`. Cet audit vérifie pour chaque partition la cohérence entre les images et leurs fichiers de labels, détecte les doublons, les fichiers corrompus et les annotations géométriquement invalides.

TABLE 3.2 – Résultats de l'audit structurel du dataset brut

Partition	Images	Labels	Manquants	Orphelins	Corrompus	Invalides
train	1 375	1 375	0	0	0	**2**
eval	153	153	0	0	0	0
test	100	100	0	0	0	0
external	180	180	0	0	0	**1**

Le dataset est structurellement propre : aucune image corrompue, aucun label manquant, aucun doublon. Seules **3 annotations invalides** ont été détectées sur l'ensemble du corpus, toutes présentant la même anomalie : une boîte englobante avec une largeur nulle (`width = 0.0`). Ce type d'annotation est probablement le résultat d'un clic accidentel lors de l'étiquetage manuel. Ces trois lignes ont été filtrées lors de la préparation sans impacter les images correspondantes.

Ces résultats confirment que le corpus est fiable comme base d'entraînement. C'est une information importante : beaucoup de projets ML échouent non pas à cause du modèle, mais à cause de données silencieusement corrompues qui biaisent l'entraînement sans que personne ne s'en rende compte.

3.3 La taxonomie initiale et ses limites

Le dataset brut utilise **14 classes d'annotation**. En les examinant, nous avons rapidement identifié deux problèmes sérieux.

Le premier est un problème de représentativité. La classe *Root resorption* ne compte que 5 annotations au total sur l'ensemble du corpus — et zéro dans les partitions eval, test et external. Un modèle ne peut pas apprendre à détecter quelque chose qu'il voit 5 fois pendant l'entraînement. Cette classe est statistiquement inexploitable en l'état.

Le second est un problème de cohérence sémantique. Plusieurs classes décrivent des états cliniquement similaires qui ont été annotés séparément. Par exemple, *Obturation*, *Endodontic treatment* et *Prosthetic restoration* désignent toutes des dents qui ont reçu un traitement. Les annoter séparément fragmente inutilement les exemples disponibles pour chaque catégorie.

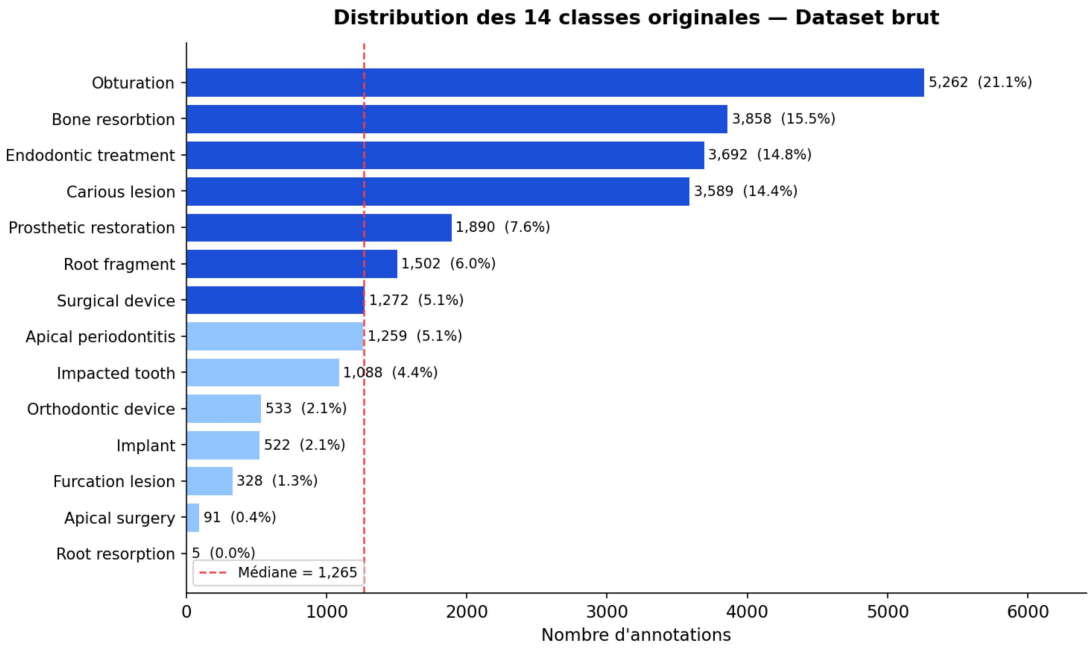


FIGURE 3.1 – Distribution des 14 classes originales — déséquilibre sévère entre les classes

La figure 3.1 illustre ce déséquilibre. Les quatre classes les plus fréquentes concentrent plus de 67% des annotations, tandis que les quatre moins représentées en cumulent moins de 2%. Un modèle entraîné sur cette distribution non corrigée développerait un biais prononcé vers les classes majoritaires.

3.4 Restructuration vers 7 classes cliniques

Pour résoudre ces problèmes, nous avons conçu un remappage de 14 classes vers **7 classes cliniques** plus cohérentes. Ce remappage a été guidé par un raisonnement clinique simple : regrouper ce qui peut être regroupé sans perdre d'information diagnostique utile.

TABLE 3.3 – Remappage des 14 classes originales vers 7 classes finales

Classes originales regroupées	→	**Classe finale**
Carious lesion	→	`CARIES`
Apical periodontitis	→	`PERIAPICAL_PATHOLOGY`
Bone resorption, Furcation lesion	→	`PERIODONTAL_BONE`
Impacted tooth	→	`IMPACTED_TOOTH`
Root fragment, Root resorption	→	`ROOT_PATHOLOGY`
Prosthetic restoration, Obturation, Endodontic treatment, Apical surgery	→	`TREATED_TOOTH`
Implant, Orthodontic device, Surgical device	→	`DEVICE_IMPLANT`

Ce remappage est implémenté dans le module `src/detection_dentaire/data/remap.py`. Il s'applique ligne par ligne lors de la lecture des fichiers de labels, en transformant chaque identifiant de classe original en son équivalent dans la nouvelle taxonomie. Les coordonnées des boîtes englobantes restent inchangées.

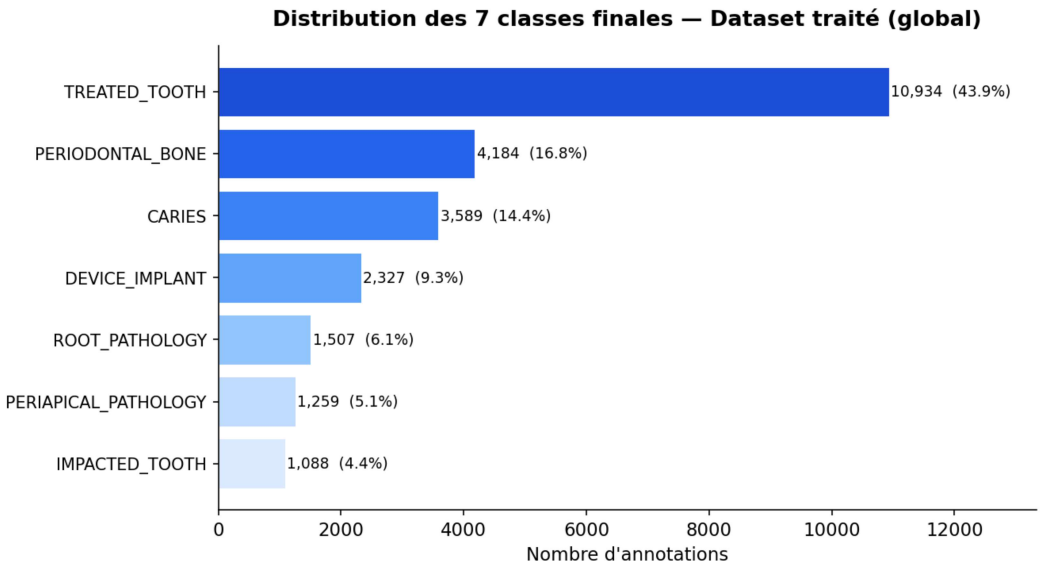


FIGURE 3.2 – Distribution après remappage — 7 classes plus équilibrées

Après remappage (figure 3.2), toutes les classes disposent d'un volume d'exemples exploitable. La classe `TREATED_TOOTH` reste dominante avec 43.9% du corpus, ce qui est

cohérent avec la réalité clinique : les dents traitées sont les structures les plus visibles et les plus fréquentes sur une radiographie panoramique adulte.

3.5 Le pipeline DVC

Toute cette chaîne de transformation — audit, remappage, génération des statistiques — est formalisée dans un pipeline DVC défini dans le fichier `dvc.yaml`. DVC (*Data Version Control*) est un outil qui étend Git aux données et aux pipelines de transformation. Il permet de définir des étapes avec leurs dépendances et leurs sorties, et de rejouer automatiquement uniquement les étapes dont les entrées ont changé.

Notre pipeline comporte trois étapes séquentielles :

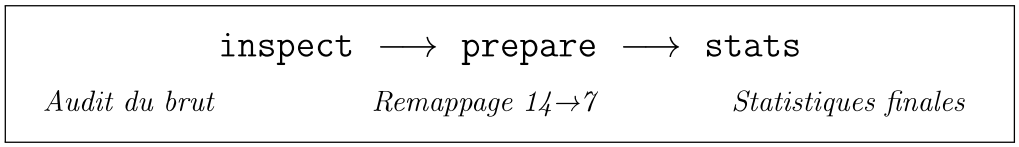


FIGURE 3.3 – Pipeline DVC du projet — trois étapes reproductibles

Étape `inspect` : lit le dataset brut, vérifie la cohérence image-label, détecte les annotations invalides et produit un rapport JSON (`reports/summaries/inspect_report.json`).

Étape `prepare` : applique le remappage des classes, filtre les annotations invalides, copie les images dans la structure cible et génère les fichiers de documentation du dataset traité.

Étape `stats` : calcule les statistiques de distribution par classe pour le dataset traité et exporte les résultats en CSV et JSON.

Pour reproduire l'intégralité de cette chaîne, une seule commande suffit : `dvc repro`. DVC détecte automatiquement quelles étapes doivent être réexécutées en fonction des modifications apportées aux données ou au code. C'est ce mécanisme qui garantit la reproductibilité du projet : n'importe qui partant du même dataset brut obtiendra exactement le même dataset traité.

3.6 Résultat : un dataset propre et documenté

À l'issue de cette phase de préparation, le dataset traité présente les caractéristiques suivantes :

TABLE 3.4 – Comparaison dataset brut / dataset traité

Version	Images	Annotations	Classes	Labels invalides
Dataset brut	1 808	24 891	14	3
Dataset traité	1 808	24 888	7	**0**

Aucune image n'a été perdue. Les 3 annotations invalides ont été filtrées. Le dataset traité ne contient aucune erreur structurelle — ce que confirme un second passage de l'outil d'audit après préparation. Il est maintenant prêt pour l'entraînement.

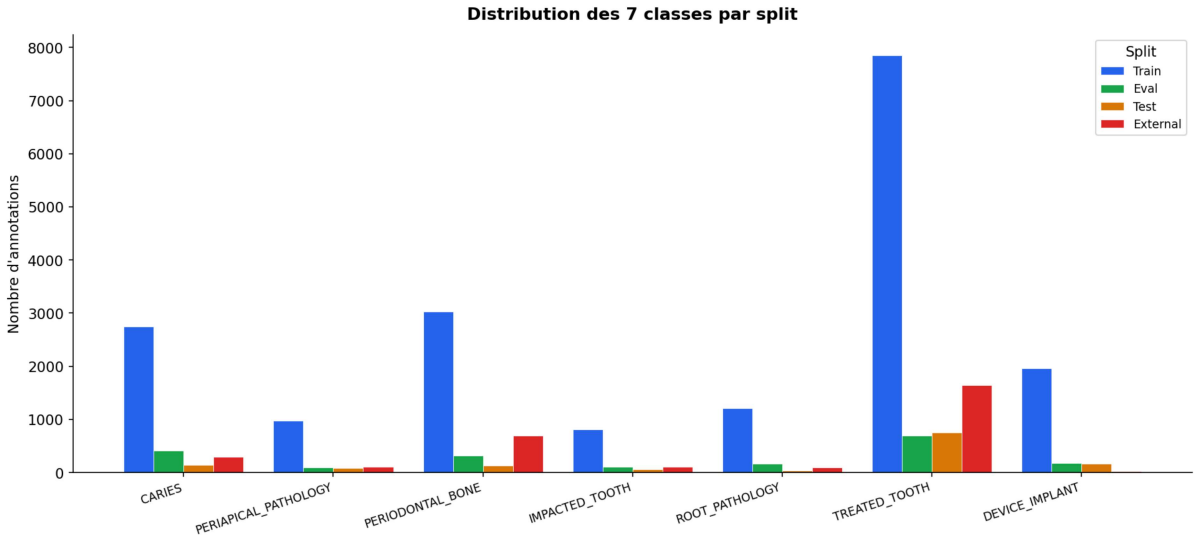


FIGURE 3.4 – Distribution des 7 classes par partition dans le dataset traité

La figure 3.4 montre que la distribution des classes est globalement cohérente entre les partitions train, eval et test. La partition `external` se distingue légèrement, avec une sur-représentation de `PERIODONTAL_BONE` et une quasi-absence de `DEVICE_IMPLANT`. Cette différence reflète les pratiques du cabinet externe et constitue précisément pourquoi ce split est un test de généralisation sérieux.

Chapitre 4

Modélisation et Expérimentation

4.1 Approche expérimentale

Dans un projet MLOps, l'entraînement d'un modèle ne se fait pas en une seule fois. On commence par une configuration de référence, on mesure les résultats, on identifie les faiblesses, on ajuste, on recommence. Ce cycle d'itération est au cœur de la phase d'expérimentation, et il ne peut être conduit sérieusement que si chaque essai est traçable et comparable.

Pour ce projet, trois variantes de modèle ont été entraînées successivement, chacune correspondant à une stratégie différente. La comparaison de ces variantes a conduit à la sélection d'un modèle *Champion* — le terme utilisé dans notre registre de modèles pour désigner le modèle actuellement en production.

4.2 Configuration de l'entraînement

Choix de la variante YOLOv8

YOLOv8 est disponible en cinq tailles, du plus léger (*nano*) au plus puissant (*xlarge*). Pour ce projet, nous avons retenu la variante **yolov8s** (*small*), qui représente 11.2 millions de paramètres. Ce choix repose sur un raisonnement pratique : avec un corpus de 1 808 images, une architecture trop grande aurait tendance à sur-apprendre. La variante *small* offre suffisamment de capacité pour apprendre les patterns visuels des anomalies dentaires, tout en restant entraînable en un temps raisonnable sur GPU.

Tous les entraînements partent du checkpoint pré-entraîné `yolov8s.pt` fourni par Ultralytics, lui-même entraîné sur le dataset COCO. Ce *transfer learning* est essentiel avec un corpus de taille limitée : le modèle bénéficie de représentations visuelles génériques déjà apprises sur des millions d'images, et se concentre sur l'adaptation à notre tâche spécifique.

Paramètres clés

TABLE 4.1 – Principaux hyperparamètres d'entraînement

Paramètre	Valeur	Justification
Résolution d'entrée	832 px	Préserver les petites lésions au redimensionnement
Optimiseur	AdamW	Meilleure convergence que SGD en fine-tuning
Taux d'apprentissage	0.0005	Faible pour un modèle pré-entraîné
Patience	35 epochs	Arrêt anticipé si pas d'amélioration
Flip horizontal	0.5	Seule augmentation agressive — médicalement cohérente
Flip vertical	0.0	Désactivé — une radio à l'envers n'a pas de sens
Augmentation couleur	Désactivée	Les radiographies sont en niveaux de gris

Le choix des augmentations mérite d'être expliqué. Dans un projet de vision par ordinateur classique, les augmentations agressives (rotations fortes, distorsions, changements de couleur) sont courantes. En imagerie médicale, cette approche est risquée : une transformation non réaliste peut introduire des artefacts visuels qui n'existent pas en conditions réelles, et dégrader les performances au lieu de les améliorer. Nous avons donc limité les augmentations aux transformations médicalement plausibles.

4.3 Le suivi des expériences avec MLflow

Chaque entraînement est tracé dans MLflow, configuré pour fonctionner sur un serveur local pendant le développement. Pour chaque *run*, MLflow enregistre automatiquement l'ensemble des paramètres, les métriques d'évaluation par partition et les artéfacts produits.

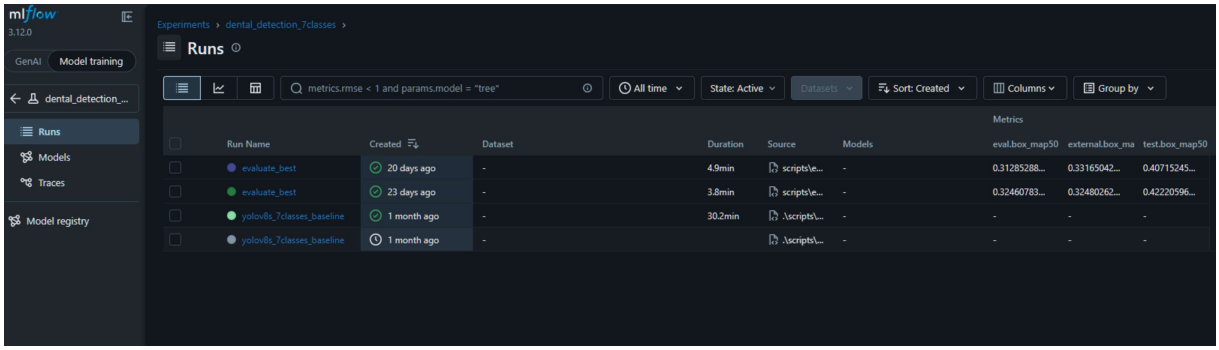


FIGURE 4.1 – Interface MLflow — liste des runs avec métriques comparées

La figure 4.1 montre l'interface MLflow avec les runs enregistrés. On y voit directement, pour chaque expérience, les valeurs de `mAP50` sur les trois partitions d'évaluation. Cette vue comparative est précisément ce qui permet de prendre une décision informée sur le choix du modèle final — sans avoir à ouvrir chaque résultat individuellement.



FIGURE 4.2 – Détail d'un run MLflow — 74 paramètres traçables

La figure 4.2 illustre la profondeur de la traçabilité : 74 paramètres sont enregistrés pour chaque run, incluant le nom du projet, la seed globale, les chemins des données, les 7 classes cibles et la table de remappage complète. Cette exhaustivité garantit qu'un run peut être reproduit à l'identique des mois plus tard, même si la configuration locale a changé.

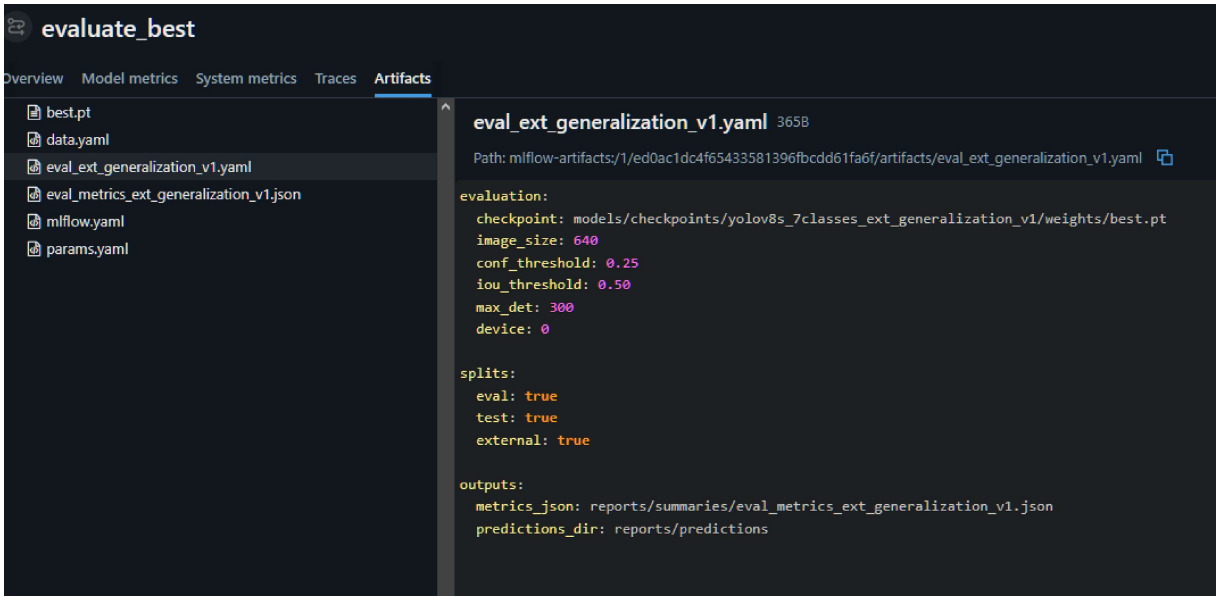


FIGURE 4.3 – Artéfacts loggés pour un run — checkpoint, configurations et métriques

4.4 Résultats et comparaison des variantes

Trois variantes ont été entraînées et évaluées sur les trois partitions de test. Les métriques retenues sont le **mAP50** (précision moyenne à un seuil IoU de 0.50), le **mAP50-95** (plus exigeant, moyenne sur plusieurs seuils IoU), la **précision** et le **rappel**.

TABLE 4.2 – Comparaison des deux variantes principales sur les trois partitions

Modèle	Partition	Précision	Rappel	mAP50	mAP50-95
Baseline	eval	0.695	0.393	0.325	0.136
Baseline	test	0.860	0.467	**0.422**	**0.211**
Baseline	external	0.682	0.383	0.325	0.146
Candidate (ext.)	eval	0.689	0.368	0.313	0.138
Candidate (ext.)	test	0.894	0.441	0.407	0.199
Candidate (ext.)	external	**0.757**	0.383	**0.332**	**0.160**

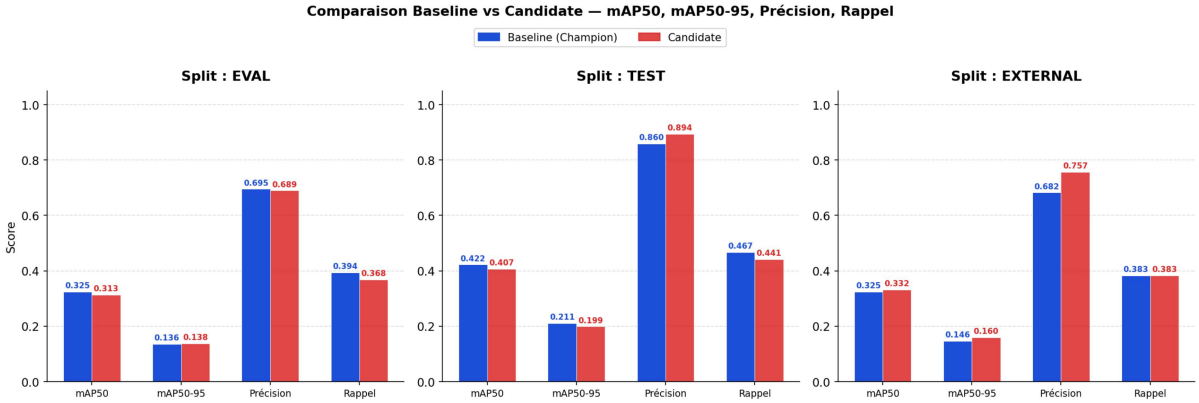


FIGURE 4.4 – Comparaison visuelle Baseline vs Candidate sur les trois partitions

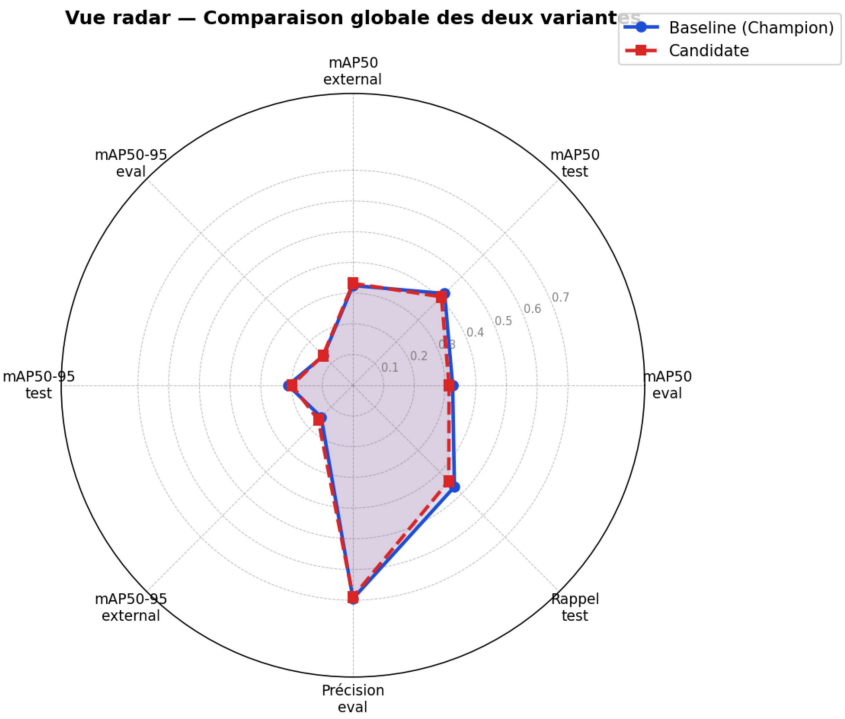


FIGURE 4.5 – Vue radar synthétique — comparaison globale des deux variantes

Les figures 4.4 et 4.5 rendent visible ce que le tableau exprime en chiffres. La situation est typique en pratique ML : aucun modèle n'est strictement supérieur sur toutes les dimensions.

La variante Candidate améliore la précision sur la partition externe (+0.075 en précision, +0.013 en mAP50-95), ce qui suggère une meilleure généralisation vers des environnements cliniques distincts. En revanche, elle recule sur les partitions internes eval et test, particulièrement sur le rappel (−0.026 sur test).

4.5 Sélection du modèle Champion

La décision de sélectionner le modèle Baseline comme **Champion** repose sur une logique de compromis global. En production, un service doit se comporter de façon cohérente dans tous les contextes, pas seulement dans le meilleur des cas. Le Baseline obtient les meilleurs scores sur les partitions test et eval, reste très compétitif sur la partition externe, et présente un rappel supérieur — ce qui est cliniquement important : manquer une anomalie est généralement plus grave que signaler une zone qui s'avère normale.

Cette décision est formalisée dans un **registre de modèles**, maintenu dans le fichier `configs/model_registry.yaml`. Ce registre définit trois rôles pour les modèles du projet :

- **Champion** : le modèle actuellement en production, exposé via l'alias stable `models/checkpoints/champion/weights/best.pt`. Tous les composants du service pointent vers cet alias.
- **Candidate** : un modèle en cours d'évaluation, potentiellement appelé à remplacer le Champion si ses performances s'avèrent supérieures sur un ensemble de critères défini.

— **Archived** : un modèle retiré de la compétition mais conservé pour la traçabilité historique.

L'état actuel du registre est le suivant :

TABLE 4.3 – État du registre de modèles

Rôle	**Modèle**	**Justification**
Champion	`yolov8s_7classes_baseline`	Meilleur compromis global eval + test
Candidate	`yolov8s_7classes_ext_gen_v1`	Meilleur sur external, moins bon globalement
Archived	`yolov8s_7classes_v2_small`	Expérimentation petites lésions, archivé

La promotion d'un checkpoint en Champion s'effectue via une commande CLI dédiée qui copie le fichier vers l'alias stable, met à jour le registre et tague le run MLflow correspondant. Ce mécanisme garantit que changer de modèle en production ne nécessite aucune modification du code du service.

Chapitre 5

Déploiement et Industrialisation

5.1 De l'expérience au service

Un modèle entraîné qui reste sur le disque dur de son auteur n'a aucune valeur opérationnelle. C'est une réalité que l'on oublie souvent dans les projets académiques, et c'est précisément là que le MLOps apporte sa valeur la plus concrète. Cette phase de déploiement est souvent la plus complexe, non pas techniquement, mais parce qu'elle oblige à penser le système dans son ensemble : comment le rendre accessible, comment garantir qu'il fonctionne dans des environnements qu'on ne contrôle pas, comment détecter une panne avant que les utilisateurs ne la signalent.

Pour ce projet, le déploiement a été pensé dès le départ comme une contrainte de conception et non comme une étape finale. Chaque décision d'architecture a été prise en se demandant : *est-ce que ça fonctionnera en production ?*

5.2 L'API REST

Conception du service

Le modèle Champion est exposé via une API REST construite avec FastAPI. Ce choix s'impose naturellement : FastAPI est aujourd'hui le framework Python le plus performant pour ce type de service, avec une génération automatique de documentation interactive et une validation native des paramètres d'entrée.

Le service est défini dans `src/detection_dentaire/serving/api.py` et expose quatre endpoints :

Table 5.1 – Endpoints de l'API REST

Méthode	Endpoint	Description
GET	`/`	Message de bienvenue et liens vers les autres endpoints
GET	`/health`	Vérification de santé : statut du service, existence du checkpoint, état du modèle
GET	`/model-info`	Informations sur le modèle Champion actif
POST	`/predict`	Analyse d'une radiographie : accepte une image en multipart/form-data, retourne les détections en JSON

Une décision de conception mérite d'être soulignée : le modèle n'est chargé en mémoire qu'à la première requête d'inférence, pas au démarrage du service. Ce *lazy loading* permet au service de répondre immédiatement aux requêtes `/health` et `/model-info` même

si le chargement du modèle prend quelques secondes. En production, cela évite qu'un orchestrateur comme Railway considère le service comme inactif pendant l'initialisation.

Format de réponse

Le endpoint `POST /predict` retourne un objet JSON structuré contenant pour chaque anomalie détectée : son identifiant de classe, son nom interprétable, son score de confiance et les coordonnées de sa boîte englobante au format $[x_1, y_1, x_2, y_2]$ en pixels absolus.

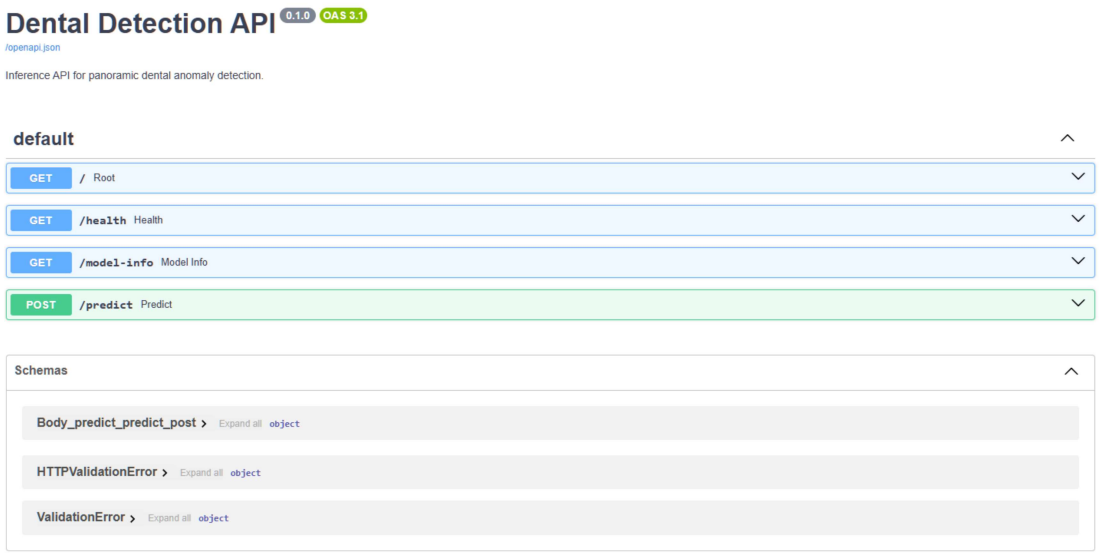


FIGURE 5.1 – Documentation Swagger de l'API — interface de test interactive sur `/docs`

FastAPI génère automatiquement une interface Swagger accessible sur `/docs`, qui permet de tester chaque endpoint directement depuis le navigateur sans outil externe. Cette documentation est maintenue automatiquement synchronisée avec le code — si un paramètre change, la documentation change avec lui.

5.3 L'interface utilisateur

Pour rendre le système accessible à un non-développeur, une interface web a été développée dans React 19 avec Vite. Elle transforme le service d'inférence en outil de démonstration interactif, utilisable sans aucune connaissance technique.

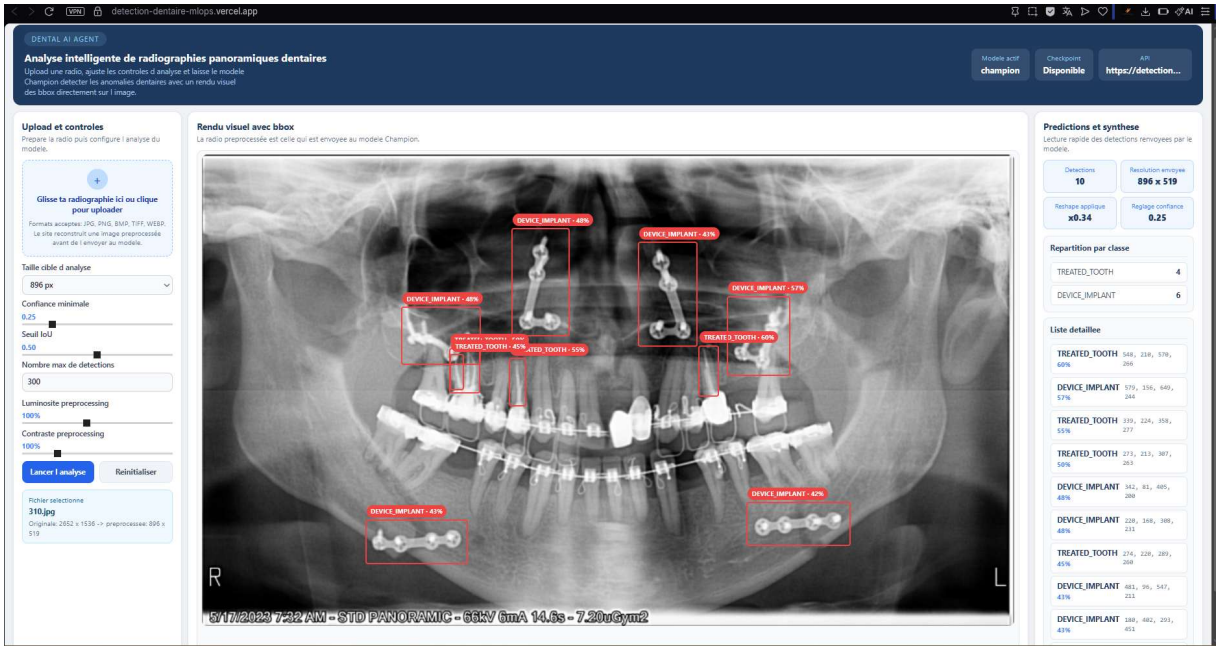


FIGURE 5.2 – Interface de démonstration — visualisation des anomalies détectées

L'interface offre plusieurs fonctionnalités pensées pour un contexte clinique : le praticien peut ajuster le seuil de confiance pour filtrer les détections peu certaines, modifier la résolution d'analyse, et régler la luminosité et le contraste de la radiographie avant envoi au modèle — ce qui améliore les performances sur les clichés sous-exposés ou surexposés.

Une décision technique importante : le **préprocessing de l'image s'effectue entièrement côté navigateur**, via un canvas HTML5, avant l'envoi au serveur. Cela réduit la bande passante consommée (une radiographie en haute résolution peut peser plusieurs mégaoctets), accélère le temps de réponse et évite de stocker des images médicales sur les serveurs.

5.4 La conteneurisation avec Docker

Pourquoi Docker est indispensable ici

Avant Docker, déployer un service Python en production était un exercice délicat. Les dépendances devaient être installées manuellement sur le serveur cible, les versions pouvaient entrer en conflit, et le comportement du service pouvait différer entre la machine de développement et le serveur de production. Docker résout ce problème en empaquetant le service, ses dépendances et son environnement d'exécution dans une image portable et reproductible.

Notre image Docker

L'image est définie dans le `Dockerfile` à la racine du projet. Nous avons fait un choix délibéré : utiliser une image CPU uniquement, basée sur `python:3.11-slim`. Cette décision réduit considérablement la taille de l'image — de 3.74 GB pour une image avec CUDA à **1.19 GB** pour notre image CPU — tout en étant parfaitement adaptée à un usage de démonstration ou de pré-production sans GPU dédié.

L'image intègre directement le checkpoint Champion (`best.pt`, 21.5 MB). Ce choix rend l'image entièrement autonome : elle peut être lancée sur n'importe quelle machine disposant de Docker sans aucune configuration supplémentaire.

TABLE 5.2 – Deux images Docker du projet — évolution de l'approche

Image	Tag	Taille	Contenu
`dental-detection-api`	`cpu`	3.74 GB	Ancienne image avec PyTorch CUDA complet
`dental-detection-api`	`latest`	1.19 GB	Image CPU optimisée avec checkpoint intégré

Pour lancer le service localement en une seule commande :

```
docker run -p 8000:8000 dental-detection-api:latest
```

5.5 Les pipelines CI/CD

Le problème que CI/CD résout

Sans automatisation, la vérification de la qualité d'un service avant chaque déploiement repose entièrement sur la vigilance humaine. Dans la pratique, ça signifie que des bugs passent en production — soit parce qu'on a oublié de tester, soit parce qu'une modification en apparence mineure a cassé quelque chose d'inattendu. Les pipelines CI/CD automatisent ces vérifications et garantissent qu'aucune modification ne peut atteindre la production sans avoir passé un ensemble de tests prédéfinis.

Notre pipeline

Deux workflows GitHub Actions ont été configurés dans `.github/workflows/`.

Le workflow **CI** (`ci.yml`) se déclenche à chaque push sur les branches principales. Il installe les dépendances dans un environnement propre, exécute les 11 tests unitaires, et vérifie que les scripts CLI principaux fonctionnent correctement. Ce pipeline s'exécute en moins de deux minutes.

Le workflow **CD** (`cd-service.yml`) est plus complet. Il enchaîne cinq jobs séquentiels :

```
unit-tests → build → integration-tests → verify-production →
                                          pipeline-summary
```

FIGURE 5.3 – Enchaînement des jobs du pipeline CD

1. **unit-tests** : les tests unitaires doivent passer avant que quoi que ce soit d'autre ne soit fait ;
2. **build** : construction de l'image Docker avec mise en cache des layers pour accélérer les builds suivants ;
3. **integration-tests** : le conteneur est démarré et les cinq endpoints sont testés automatiquement avec des assertions Python ;

4. **verify-production** : les services déployés sur Railway et Vercel sont vérifiés directement, confirmant que la production est opérationnelle ;
5. **pipeline-summary** : rapport final avec le statut de chaque job et les URLs de production.

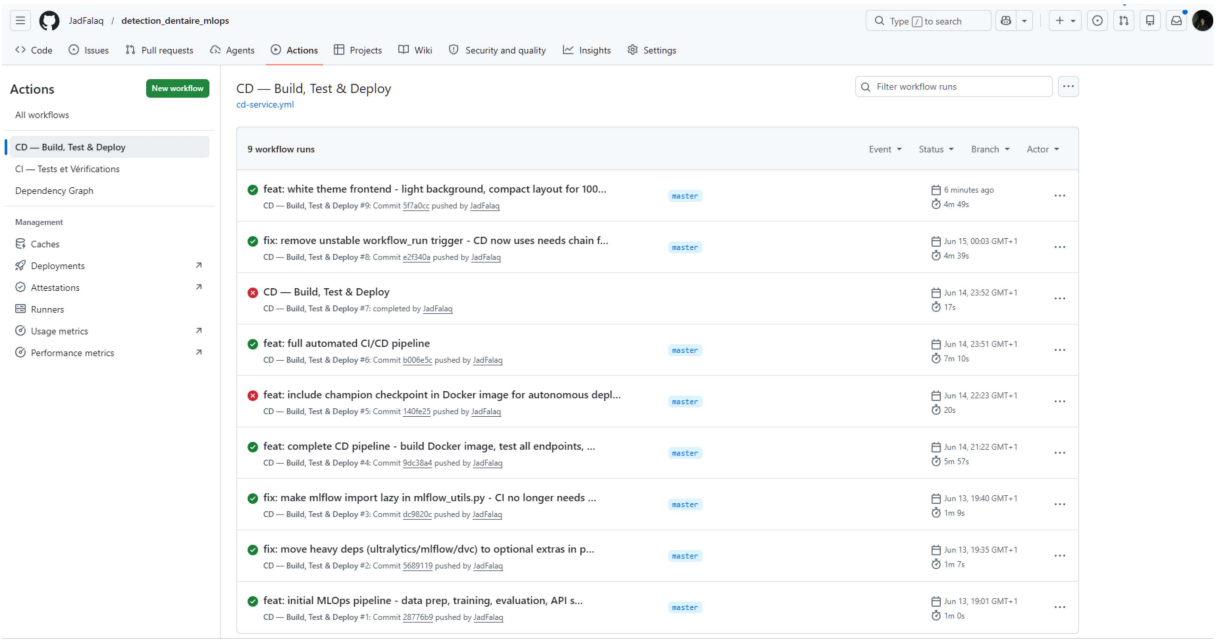


FIGURE 5.4 – Pipeline CD en action — tous les jobs au vert

Les tests d'intégration en détail

La partie la plus intéressante du pipeline CD est sans doute les tests d'intégration. Contrairement aux tests unitaires qui testent des fonctions isolées, ces tests lancent réellement le conteneur Docker et vérifient le comportement du service de bout en bout. Pour chaque endpoint, une assertion Python vérifie la structure et le contenu de la réponse.

Le endpoint `/health` est testé en priorité : il doit retourner `status: ok` et confirmer que le checkpoint existe dans le conteneur. C'est le test le plus critique — si ce test échoue, le service ne peut pas fonctionner et le pipeline s'arrête immédiatement.

5.6 L'infrastructure cloud

Backend sur Railway

Railway est une plateforme cloud qui permet de déployer des applications conteneurisées directement depuis un dépôt GitHub. À chaque push sur la branche `master` qui modifie les fichiers du service, Railway reconstruit et redéploie automatiquement l'image Docker. Le service est accessible à l'adresse :

`https://detectiondentairemlops-production.up.railway.app`

Une vérification directe de l'état du service en production confirme son bon fonctionnement :

```
{
  "status": "ok",
  "service": "dental-detection-api",
  "model_name": "champion",
  "checkpoint_exists": true,
  "model_loaded": true
}
```


Frontend sur Vercel

L'application React est déployée sur Vercel, une plateforme spécialisée dans le déploiement d'applications JavaScript modernes. Comme Railway pour le backend, Vercel redéploie automatiquement le frontend à chaque modification du dossier `frontend/` dans le dépôt. L'interface est accessible à l'adresse :

`https://detection-dentaire-mlops.vercel.app`

Vue d'ensemble de l'architecture déployée

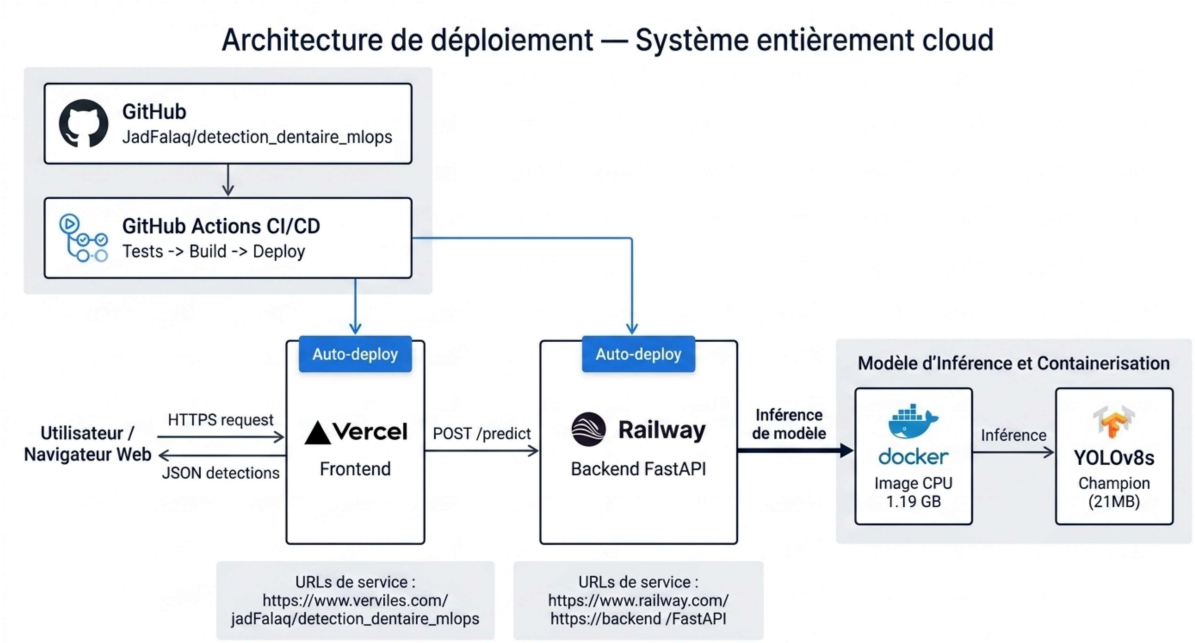


FIGURE 5.5 – Architecture de déploiement — système entièrement cloud

L'architecture finale est entièrement cloud. L'utilisateur accède à l'interface Vercel depuis son navigateur, qui envoie les requêtes d'analyse au backend Railway. Le backend analyse la radiographie avec le modèle Champion embarqué dans l'image Docker et retourne les détections en JSON. L'interface affiche les boîtes englobantes superposées sur l'image. Aucun composant ne dépend d'une machine locale.

Liens du projet

L'ensemble des ressources du projet sont accessibles publiquement aux adresses suivantes :

TABLE 5.3 – Liens de déploiement et code source du projet

Ressource	**URL**
Code source (GitHub)	`https://github.com/JadFalaq/detection_dentaire_mlops`
Interface utilisateur (Vercel)	`https://detection-dentaire-mlops.vercel.app`
API backend (Railway)	`https://detectiondentairemlops-production.up.railway.app`
Documentation API (Swagger)	`https://detectiondentairemlops-production.up.railway.app/docs`

Chapitre 6

Conclusion et Perspectives

6.1 Ce que ce projet a réellement accompli

Au départ, l'objectif était simple à énoncer : construire un système de détection d'anomalies dentaires et le déployer. Mais entre l'énoncé et la réalisation, il y a eu un travail d'ingénierie considérable que ce rapport a tenté de documenter honnêtement.

Le résultat est un service en ligne, fonctionnel, accessible publiquement, qui prend en entrée une radiographie panoramique et retourne les anomalies détectées avec leur localisation. Ce service n'est pas une démonstration de laboratoire : il tourne sur une infrastructure cloud réelle, avec un modèle entraîné sur des données cliniques annotées, un pipeline de tests automatisés et une chaîne de déploiement continue.

Table 6.1 – Bilan des réalisations par dimension MLOps

Dimension	**Ce qui a été fait**	**Outil utilisé**
Données	Audit, nettoyage, remappage 14→7 classes, pipeline reproductible	DVC
Expérimentation	3 variantes entraînées, toutes tracées et comparables	MLflow
Sélection	Registre Champion/Candidate/Archived, alias stable en production	`model_registry.yaml`
Service	API REST documentée, 4 endpoints, chargement paresseux du modèle	FastAPI
Portabilité	Image Docker CPU autonome avec modèle intégré, 1.19 GB	Docker
Automatisation	11 tests unitaires, 5 tests d'intégration, vérification production	GitHub Actions
Déploiement	Service cloud public, redéploiement automatique sur push	Railway + Vercel

6.2 Ce qui a bien fonctionné

Plusieurs décisions prises en début de projet se sont révélées particulièrement payantes.

Penser le déploiement dès le début. En concevant l'API et la structure modulaire du code avant même de lancer le premier entraînement, nous avons évité le piège classique du "on verra pour le déploiement à la fin". Cette anticipation a rendu la transition vers la production beaucoup plus fluide.

Intégrer le checkpoint dans l'image Docker. Cette décision, prise après avoir constaté que Railway ne pouvait pas accéder aux fichiers locaux, a rendu l'image entièrement autonome. Une commande suffit pour lancer le service complet sur n'importe quelle machine.

Utiliser un registre de modèles léger. Un simple fichier YAML maintenant l'état du registre Champion/Candidate/Archived s'est avéré suffisant pour ce projet. Il a permis de changer de modèle en production sans modifier une seule ligne de code du service.

Les tests d'intégration dans le pipeline CD. Tester les vrais endpoints du conteneur Docker dans le pipeline CI/CD — et non seulement des fonctions unitaires — a permis de détecter plusieurs problèmes qui n'auraient pas été visibles autrement : le problème de séparateur de chemin Windows/Linux dans les tests du registre, le conflit entre `device=0` et l'absence de CUDA, la dépendance sur MLflow lors de l'import du module.

6.3 Les limites du système

Ce serait malhonnête de conclure sans mentionner les limites réelles.

Les performances du modèle restent modestes. Un mAP50 de 0.32 sur la partition eval signifie que le modèle rate environ 68% des détections au seuil IoU de 0.50. Cela est partiellement dû à la taille limitée du corpus (1 808 images), au déséquilibre sévère entre les classes, et à la résolution d'entrée contrainte par les ressources disponibles. Ce niveau de performance est acceptable pour une démonstration, insuffisant pour un usage clinique réel.

Pas de validation clinique. Le système n'a pas été évalué par des praticiens ni comparé à des diagnostics de référence. Les 7 classes retenues sont cliniquement motivées, mais leur pertinence réelle pour l'aide au diagnostic reste à démontrer dans un contexte médical formalisé.

Le split external révèle une fragilité. La différence de distribution entre le split external et les splits internes — notamment la quasi-absence de `DEVICE_IMPLANT` et la sur-représentation de `PERIODONTAL_BONE` — indique que le modèle est sensible aux variations de pratiques entre cabinets. Un système robuste devrait être testé sur des données provenant d'au moins cinq à dix environnements cliniques distincts.

6.4 Perspectives d'évolution

Ce projet constitue une base solide sur laquelle plusieurs évolutions naturelles peuvent être envisagées.

Enrichir le corpus. C'est la priorité absolue pour améliorer les performances. Doubler la taille du dataset permettrait probablement de gagner 5 à 10 points de mAP50,

sans changer l'architecture. L'ajout de données provenant de plusieurs cabinets distincts améliorerait la robustesse de généralisation.

Entraîner avec des résolutions plus élevées. Nos expériences ont utilisé 832 pixels comme résolution d'entrée. Passer à 1 024 ou 1 280 pixels, avec un GPU plus puissant, permettrait de mieux capturer les petites lésions dont l'aire normalisée est inférieure à 1% de l'image.

Ajouter un tableau de bord de monitoring. En production, il serait précieux de tracer le temps de réponse de l'API, le nombre de requêtes quotidiennes et les distributions des classes détectées. Ces métriques opérationnelles permettent de détecter une dérive du modèle avant qu'elle ne devienne problématique.

Implémenter un vrai model registry opérationnel. Le registre basé sur un fichier YAML est suffisant pour un projet académique, mais une intégration complète avec MLflow Model Registry permettrait de gérer les versions de modèles avec des métadonnées plus riches et une interface de comparaison intégrée.

Validation clinique. L'étape qui transformerait ce projet en outil réellement utile est une évaluation par des chirurgiens-dentistes, comparant les détections du modèle à leurs propres lectures. Cette validation fournirait des métriques cliniquement significatives — sensibilité, spécificité — bien plus parlantes pour un usage médical que le mAP50.

6.5 Réflexion finale

Ce projet a confirmé quelque chose que les cours de MLOps énoncent mais qu'on ne comprend vraiment qu'en pratique : la valeur d'un système ML ne réside pas dans la performance brute de son modèle, mais dans sa capacité à fonctionner de façon fiable, à évoluer sans régressions et à être utilisé par quelqu'un d'autre que son auteur.

L'entraînement de YOLOv8 a été la partie la plus simple de ce travail. Ce qui a pris le plus de temps — et qui a apporté le plus de valeur — c'est tout ce qui l'entoure : comprendre les données avant de les utiliser, tracer les expériences pour pouvoir les comparer, concevoir un service qui se comporte correctement en production, automatiser les vérifications pour ne pas regretter d'avoir poussé une modification la veille d'une démonstration.

C'est précisément cela que le MLOps cherche à enseigner, et c'est ce que ce projet a, dans une large mesure, réussi à mettre en pratique.

Bibliographie

[1] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics YOLOv8. `https://github.com/ultralytics/ultralytics`, 2023. Version 8.0.0. Accès : 2025.

[2] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once : Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016.

[3] David Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems (NIPS)*, volume 28, pages 2503–2511. Curran Associates, Inc., 2015.